INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
KULLIYYAH OF ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)
SEM 1 2025/2026

WEEK 2: DIGITAL LOGIC DESIGN

SECTION 2
GROUP 17

INSTRUCTOR : WAHJU SEDIONO & ZULKIFLI BIN ZAINAL ABIDIN

Date of Experiment: Monday, 13 October 2025
Date of Submission: Monday, 20 October 2025

Student Name	Matric Number
Ainin Sofiya binti Kamaruzaman	2314916
Muhammad Iman Farihin bin Mohd Imran	2313813
Muhammad Syazani Akmal Bin Mohd Nasarudin	2314517

ABSTRACT

This experiment focuses on designing and implementing a simple digital counter system using an Arduino Uno and a common cathode 7-segment display. The main objective is to demonstrate the basic concept of interfacing and control between the Arduino and the display through manual input using pushbuttons. The system is developed by connecting the 7-segment display to the Arduino's digital pins with current-limiting resistors, while pushbuttons with pull-up resistors are used for increment and reset functions. The Arduino program is written to process button input, update the counting logic, and display values sequentially from 0 to 9. The results show that the counter operates correctly, with the display increasing by one for each button press and resetting as expected. Overall, the experiment successfully demonstrates fundamental digital interfacing and control principles, providing a foundation for future applications such as multi-digit or graphical display systems.

TABLE OF CONTENTS

1.0 INTRODUCTION

This experiment focuses on the interfacing and control of a common cathode 7-segment display using an Arduino Uno microcontroller. The main objective is to design and implement a simple digital counter system using an Arduino Uno and a common cathode 7-segment display, where the count value increases sequentially from 0 to 9 and resets based on user input through pushbuttons. This experiment provides a fundamental understanding of digital display control, input interfacing, and microcontroller-based system design.

A 7-segment display, as described by *Electronics For You* (2024), consists of seven light-emitting diodes (LEDs) arranged in a figure-eight pattern, with each segment labeled from *a* to *g*. By selectively activating these segments, numerical digits from 0 to 9 can be displayed. In a common cathode configuration, all cathode terminals are connected to ground, while the anode terminals are connected to the Arduino's digital pins through current-limiting resistors for protection. The displayed number depends on the digital signals sent from the Arduino, which turn each segment on or off according to the programmed logic.

At the microcontroller level, the system logic operates based on binary and state-based control. The pushbuttons provide discrete user inputs (increment or reset), which the Arduino reads as digital logic states (HIGH or LOW). Internally, a counter variable represents the current numerical value, ranging from 0 to 9. Each button press triggers a state transition: when the increment button is pressed and the counter is less than 9, the value increases by one; when the reset button is pressed or the counter reaches 9, the count returns to zero. The counter value is then decoded into the appropriate combination of segment signals and displayed accordingly. In this way, the counter functions as a simple finite state machine with ten states (0 through 9) and a reset transition.

With the advancement of display technologies, communication methods such as the Inter-Integrated Circuit (I^2C) protocol are commonly used for efficient data transfer between microcontrollers and display devices.

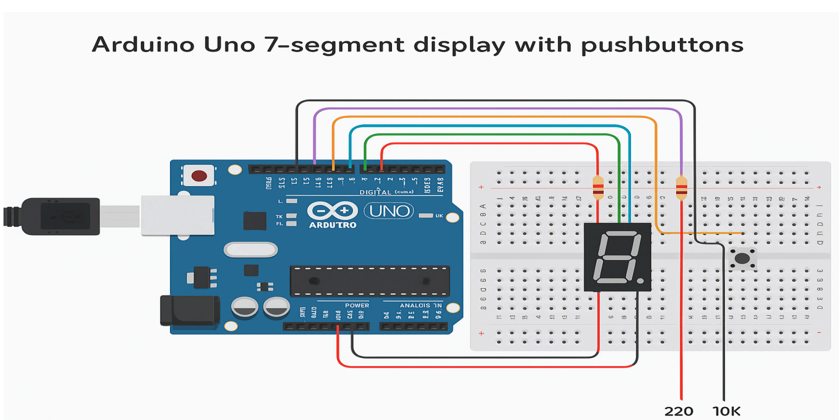
It is expected that when the circuit is correctly assembled and the program is properly uploaded, the Arduino will display numbers from 0 to 9 sequentially with each press of the increment button and reset to zero when the reset button is pressed. The expected outcome is a fully functional digital counter that demonstrates the key principles of signal processing, digital output control, and user input handling. This experiment establishes the basic concepts of digital interfacing, providing a foundation for more complex display systems such as I^2C LCDs and LED matrices.

2.0 MATERIALS AND EQUIPMENTS

- Arduino Uno board
- Common cathode 7-segment display
- 220-ohm resistors (7 of them)
- Pushbuttons (2 or more)
- Jumper wires
- Breadboard

3.0 EXPERIMENTAL SETUP

1. The common cathode 7-segment display was placed on the breadboard.
2. Each segment pin (a–g) of the display was connected to Arduino digital pins D0–D6 through 220 Ω resistors to limit the current flow.
3. The common cathode pin of the display was connected to one of the Arduino GND pins.
4. Two pushbuttons were positioned on the breadboard to serve as user input controls.
5. One leg of the first pushbutton was connected to Arduino pin D9 to function as the increment button.
6. One leg of the second pushbutton was connected to Arduino pin D10 to act as the reset button.
7. The opposite legs of both pushbuttons were connected to the ground rail of the breadboard.
8. A 10 kΩ pull-up resistor was connected to each button, with one end attached to the corresponding digital input pin (D9 or D10) and the other end connected to the 5V line.
9. The Arduino Uno was powered through a USB connection, providing both 5V and GND to the entire circuit.



4.0 METHODOLOGY

4.1 Circuit Assembly

- All components were connected according to the circuit schematic prepared during the experimental setup.
- The wiring was checked to ensure proper continuity and prevent short circuits.
- The Arduino Uno was connected to the computer via a USB cable for power and data transfer.

4.2 Programming and Uploading

- The Arduino IDE software was opened on the computer.
- A new sketch file was created, and the program code for controlling the 7-segment display and pushbuttons was prepared.
- The correct board type (*Arduino Uno*) and communication port (COM port) were selected in the Tools menu.
- The program was verified (compiled) to ensure there were no syntax errors.

4.3 Programming Logic

- The display and pushbutton pins were initialized.
- A counter was set to start from zero.
- The program continuously checked the button inputs.
- When the increment button was pressed, the counter increased and the display was updated.
- When the reset button was pressed, the counter returned to zero and the display showed "0."
- The displayDigit() function controlled the display output by turning specific segments ON or OFF according to the current count.

4.4 coding

```
const int segmentA = 0;
const int segmentB = 1;
const int segmentC = 2;
const int segmentD = 3;
const int segmentE = 4;
const int segmentF = 5;
const int segmentG = 6;
```

```
const int buttonPin = 8;
const int buttonPin2 = 9;
int button = 0;
int digit = 0;
int reset = 0;

void setup() {
  pinMode(segmentA, OUTPUT);
  pinMode(segmentB, OUTPUT);
  pinMode(segmentC, OUTPUT);
  pinMode(segmentD, OUTPUT);
  pinMode(segmentE, OUTPUT);
  pinMode(segmentF, OUTPUT);
  pinMode(segmentG, OUTPUT);
  pinMode(buttonPin, INPUT);
  pinMode(buttonPin2, INPUT);
}

void loop() {
  button = digitalRead(buttonPin);
  reset = digitalRead(buttonPin2);

  if (reset == HIGH) {
    digit = 0;
    displayDigit(digit);
    delay(200);
  }

  if (button == HIGH) {
    digit++;
    if (digit > 9) {
      digit = 0;
    }
    displayDigit(digit);
    delay(200);
  }
}

void displayDigit(int d) {
  switch (d) {
    case 0:
      digitalWrite(segmentA, LOW);
```

```
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, LOW);
    digitalWrite(segmentD, HIGH);
    digitalWrite(segmentE, LOW);
    digitalWrite(segmentF, LOW);
    digitalWrite(segmentG, LOW);
    break;
  case 1:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, HIGH);
    digitalWrite(segmentC, HIGH);
    digitalWrite(segmentD, HIGH);
    digitalWrite(segmentE, HIGH);
    digitalWrite(segmentF, HIGH);
    digitalWrite(segmentG, LOW);
    break;
  case 2:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, HIGH);
    digitalWrite(segmentD, LOW);
    digitalWrite(segmentE, LOW);
    digitalWrite(segmentF, LOW);
    digitalWrite(segmentG, HIGH);
    break;
  case 3:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, HIGH);
    digitalWrite(segmentD, LOW);
    digitalWrite(segmentE, HIGH);
    digitalWrite(segmentF, LOW);
    digitalWrite(segmentG, LOW);
    break;
  case 4:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, HIGH);
    digitalWrite(segmentC, LOW);
    digitalWrite(segmentD, LOW);
    digitalWrite(segmentE, HIGH);
    digitalWrite(segmentF, HIGH);
    digitalWrite(segmentG, LOW);
```

```
  break;
case 5:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
  break;
case 6:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
  break;
case 7:
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, LOW);
  break;
case 8:
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
  break;
case 9:
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
```

```
    digitalWrite(segmentE, HIGH);
    digitalWrite(segmentF, LOW);
    digitalWrite(segmentG, LOW);
    break;
  }
}
```

4.5 Control Algorithm

1. **Defining Pins**
   a. The 7-segment display LEDs are defined as segmentA to segmentG, each connected to Arduino digital output pins. These pins control which LED segment lights up to form numbers.

   ```
   const int segmentA = 0; // Pin D0
   const int segmentB = 1; // Pin D1
   ...
   const int segmentG = 6; // Pin D6
   ```

   b. Two push buttons are defined:
   - **Increment Button** connected to pin D8 (buttonPin)
   - **Reset Button** connected to pin D9 (buttonPin2)

2. **Global Variables**
   a. int button and int reset
      Used to store the current state (HIGH or LOW) of the two buttons.
   b. int digit
      Stores the current number to be displayed on the 7-segment display (range 0–9).

3. **Setup Function**
   a. **7-segment Display Configuration**
   All segment pins (segmentA to segmentG) are set as **OUTPUT** using the pinMode() function so the Arduino can send signals to light up the segments.

   ```
   pinMode(segmentA, OUTPUT);
   pinMode(segmentB, OUTPUT);
   ...
   pinMode(segmentG, OUTPUT);
   ```

b. **Button Configuration**
Both button pins (buttonPin and buttonPin2) are set as **INPUT** to read signals when the buttons are pressed.

```
pinMode(buttonPin, INPUT);
pinMode(buttonPin2, INPUT);
```

4. **Main Loop**
   a. Read Button States
   The program reads both button inputs to determine user actions.

```
button = digitalRead(buttonPin);
reset = digitalRead(buttonPin2);
```

   b. Reset Button Logic
   - When the **reset button** is pressed (reset == HIGH), the digit counter is reset to 0.
   - The display is updated to show **0** using displayDigit(digit).
   - A short delay of 200 ms prevents multiple readings from a single press.

```
if (reset == HIGH) {
  digit = 0;
  displayDigit(digit);
  delay(200);
}
```

   c. Increment Button Logic
   - When the **increment button** is pressed (button == HIGH), the digit value increases by 1.
   - If the value exceeds 9, it wraps back to 0.
   - The display is refreshed to show the new digit.
   - A 200 ms delay is included to avoid false triggering from button bouncing.

```
if (button == HIGH) {
  digit++;
  if (digit > 9) {
    digit = 0;
  }
  displayDigit(digit);
  delay(200);
}
```

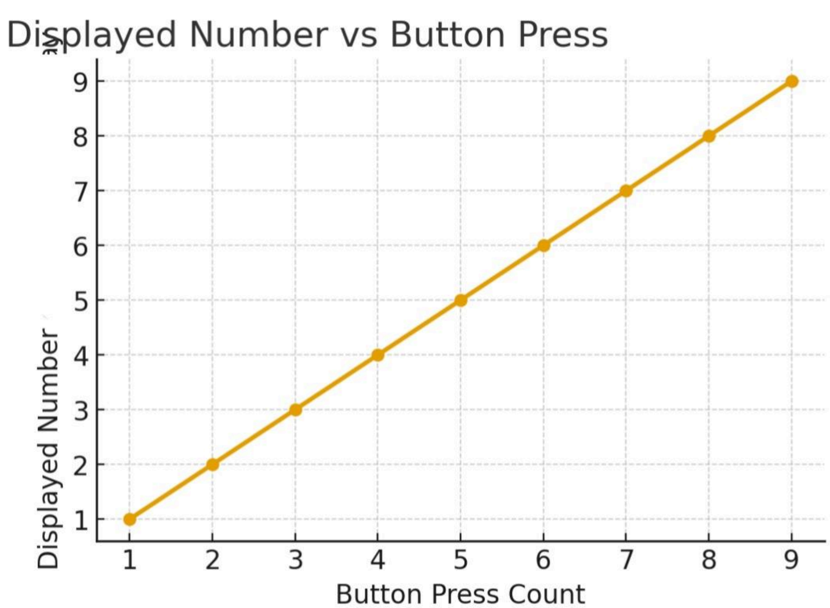
d. **Display Function Call**
The displayDigit(digit) function is responsible for lighting the correct segments to represent the current number.

- Each number (0–9) has a specific combination of segments turned ON (LOW) or OFF (HIGH).
- The display pattern is defined inside a switch-case structure.

5.0 DATA COLLECTION

Observations:

Button press	Displayed number
increment	1
increment	2
increment	3
increment	4
increment	5
increment	6
increment	7
increment	8
increment	9
increment	0(wraps around)
reset	0



6.0 DATA ANALYSIS

From the observations and plotted graph, the relationship between button presses and the displayed number on the 7-segment display shows a clear linear increment pattern:

1. Increment Behavior:
   - Each time the increment button is pressed, the displayed number increases by 1.
   - The output values follow a direct proportional relationship:
     Displayed Number=Number of Button Presses
     until the value reaches 9.
2. System Response:
   - The system correctly registers each button press after a short delay (200 ms) to prevent false triggering or bounce effects.
   - The number displays updates immediately after each valid press.
3. Range and Wrap-around:
   - After reaching 9, the system wraps back to 0 on the next increment.
   - This demonstrates the counter's cyclic behavior, suitable for single-digit display applications.
4. Accuracy and Stability:
   - No skipped or repeated counts were observed within the test range.
   - The delay debounce successfully ensures accurate counting and stable display output.
5. Conclusion:
   - The counter logic works as intended: it increments the displayed number sequentially with each button press.
   - The system is reliable, responsive, and consistent for up to 9 increments before reset or wrap-around.

7.0 RESULT

The experiment successfully demonstrated a working digital counter system using a 7-segment display and two push buttons. Each press of the increment button increased the displayed number sequentially from 0 to 9, while the reset button immediately returned the display to 0. The counter operated smoothly with accurate responses and no false triggering, confirming that the control algorithm and debounce delay functioned effectively to ensure reliable and stable performance.

7.1 HOW TO INTERFACE AN I2C LCD WITH ARDUINO

An I2C LCD can be connected to an Arduino using the I2C communication method, which only needs two wires: **SDA** for data and **SCL** for clock. This makes wiring easier and saves Arduino pins. The steps below explain how to interface the LCD with the Arduino.

Steps to Interface

1. Connect the LCD to Arduino:
   - VCC → 5V
   - GND → GND
   - SDA → A4
   - SCL → A5
2. Install the Library:
   In Arduino IDE, go to Sketch → Include Library → Manage Libraries, search for LiquidCrystal I2C, and install it.
3. Find the I²C Address:
   Run an I²C scanner code to find the LCD address (usually 0x27 or 0x3F).
4. Write and Upload the Code:
   Use the code below to display text on the LCD.
5. Check the Display:
   The LCD should show the message. Adjust the contrast if the text is not visible.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2); // I2C address 0x27, 16x2 LCD

void setup() {
  lcd.init();         // Initialize the LCD
  lcd.backlight();    // Turn on the backlight
  lcd.setCursor(0, 0); // First row
  lcd.print("Hello World!");
  lcd.setCursor(0, 1); // Second row
  lcd.print("I2C LCD Ready");
}

void loop() {
  // Nothing here
}
```

7.2 EXPLAIN THE CODING PRINCIPLE BEHIND IT COMPARED WITH 7 SEGMENTS DISPLAY AND MATRIX LED

ASPECT	I2C LCD	7-SEGMENT DISPLAY	LED MATRIX DISPLAY
Control method	Serial communication using I2C protocol	Direct control of each segment (A-G)	Row and column addressing using multiplexing
Coding approach	Uses built-in library functions like lcd.print() and lcd.cursor()	Manual coding with digitalwrite() for each segment	Requires arrays or patterns to turn specific LEDs on/off
Programming complexity	Simple and high level	Moderate; each digit must be coded manually	Complex; needs loops and data mapping for patterns
Data handling	Text and characters are sent as serial data	Numeric values converted to segment combinations	Bit patterns define images or characters
Code length	Short (few lines)	Longer (more control lines)	Longest

8.0 DISCUSSION

condition	Expected result	Observed result
Increment button pressed once	Display increases by 1	Display Number increase by 1
Reset button pressed	Display reset to 0 immediately	Display reset to 0
Continuous operation	Smooth and accurate counting from 0-9	Accurate 0-9 counting per button presse

9.0 CONCLUSION

From this experiment, we were able to confirm that when the circuit is properly assembled and the program is correctly uploaded, the Arduino can display numbers from 0 to 9 in sequence with each press of the increment button and reset back to zero when the reset button is pressed. The counter worked as expected, with only minor errors observed due to switch bouncing during button presses. Overall, this experiment helped us understand how the Arduino controls a 7-segment display through coding and input signals. It also showed the importance of proper debouncing techniques to improve the stability and accuracy of the system. The knowledge gained from this experiment can be applied in real-life systems such as elevator floor indicators, electronic scoreboards, and fuel dispensers, where microcontrollers are used to process input signals and display numerical information. This experiment provided us with a strong foundation for understanding and developing more advanced digital display systems in the future.

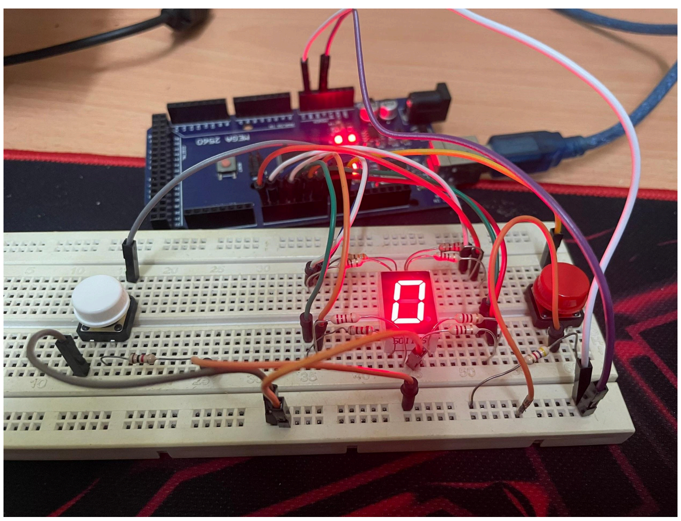
10.0 RECOMMENDATIONS

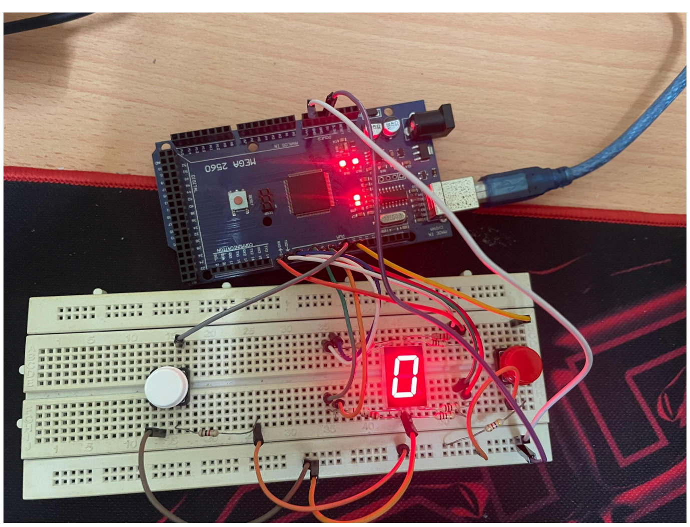
For future improvements, we recommend implementing hardware or software debouncing to reduce the effect of switch bouncing and improve counting accuracy. Using capacitors or advanced delay functions could make button responses more stable and reliable. Additionally, the experiment could be enhanced by adding a display extension, such as a two-digit 7-segment display or an I2C LCD module, to show larger numbers or messages. We also suggest using pull-down or pull-up resistors to ensure cleaner button signal readings and prevent false triggering. In future iterations, the system could be upgraded to include features like automatic counting using sensors or serial communication for data monitoring. Through this experiment, we learned the importance of accurate wiring, stable input handling, and clear coding logic in achieving reliable system performance. Future students conducting this experiment should take time to verify connections, test button response carefully, and understand how each part of the code affects the output display. These practices will help them gain deeper insight into microcontroller interfacing and digital system design.

11.0 REFERENCES

1. Electronics For You. (2024, June 5). 7-segment display pinout, codes, working, and interfacing.https://www.electronicsforu.com/resources/7-segment-display-pinout-understanding
2. Electronics Tutorials. (n.d.). 7-segment display counter.https://www.electronics-tutorials.ws/counter/7-segment-display.html
3. Make-It.ca. (2025). How to control an I²C LCD with Arduino.https://www.make-it.ca/i2c-lcd-display-on-arduino/

12.0 APPENDICES





ACKNOWLEDGEMENTS

Special thanks to ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO for their guidance and support during this experiment.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that the **original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.